

生成树算法及扩展

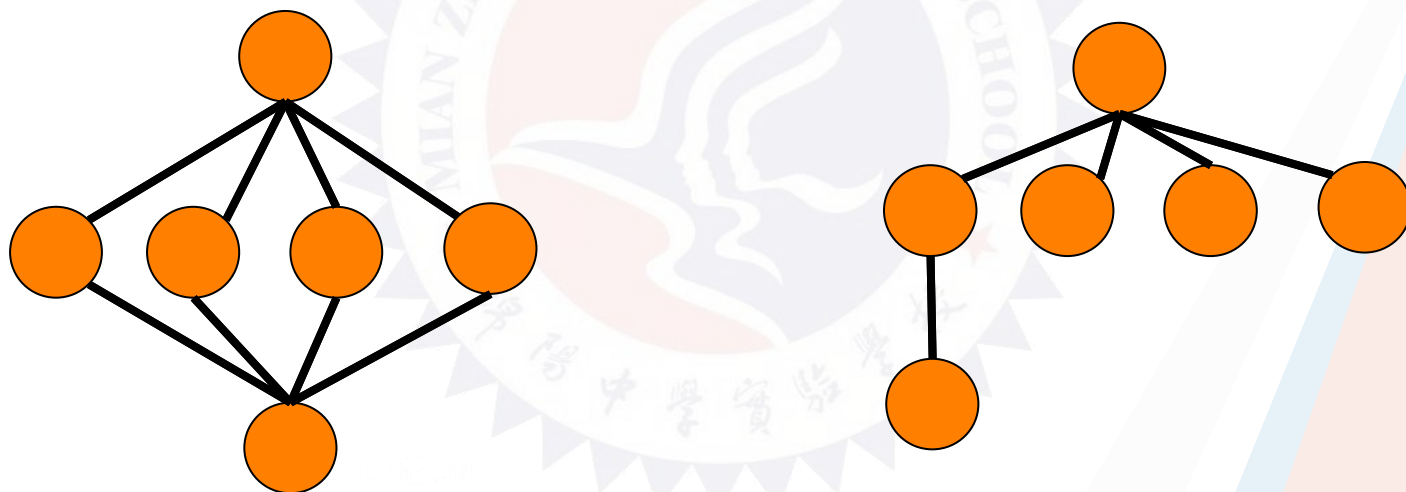
主讲：黄凤林

→ 图与树

图： n 个结点， m 条边，仅考虑点与点之间是否有相联关系。

树： n 个点， $n-1$ 条边的连通图。

生成树：从图的 m 条边中，根据一定规则选出其中 $n-1$ 条边连通，则构成的就是图的生成树。





→ 生成树的模型

生成树变形常见有以下类型：

最小生成树

次小生成树

有限度的生成树

最有比率生成树

最短路径生成树

最小生成树计数

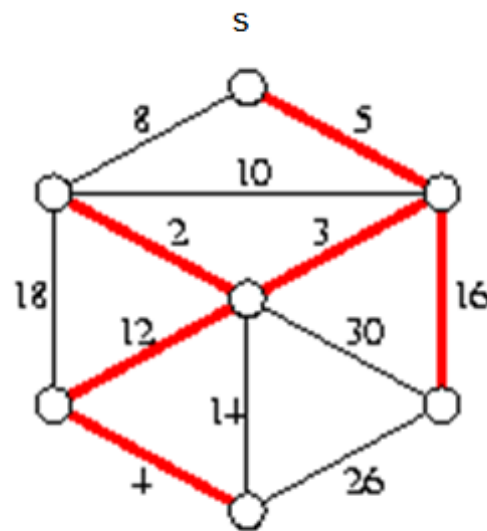
最小差值生成树

→ 最小生成树

➤ 生活背景:

要在 n 个居民点之间架设煤气管道, 很显然最多可架设 $n(n-1)/2$ 条管道, 然而要连通 n 个居民点架设 $n-1$ 条管道(无环出现)就可以了, 为了使**总造价**最小, 选择哪 $n-1$ 条边? 这就是最小生成树问题。

最小生成树: 在一个具有 n 个顶点的连通图 G 中, 如果存在子图 G' 包含 G 中所有顶点和一部分边, 且不形成回路, 则称 G' 为图 G 的生成树, **权值和最小的生成树**则称为最小生成树



最小生成树

→ 最小生成树

✧ 最小生成树算法：根据图转生成树的思路，可分两个方法

✧ 算法一： **prim**

算法思想：按点加入生成树，加入 n 个点。

时间复杂度： $O(v^2)$

✧ 算法二： **Kruskal**

算法思想：按边加入生成树,加入 $n-1$ 条边。

时间复杂度： $O(E \log E)$

→ Prim算法

基本思想：向生成树中一个一个的加点

实现步骤：

- ✧ 设图 $G=(V,E)$ ，其生成树的顶点集合为 U 。
- ✧ 1 :把 v_1 放入 U 中，并更新其他点到生成树的距离。
- ✧ 2 :找到离生成树距离最近且不在生成树集合 U 中的点 v_i ，将 v_i 加入生成树中，然后再次更新不在生成树中的点到生成树的距离。
- ✧ 3 :如果 U 集合已有 n 个元素，则结束，否则继续执行2。

注意：

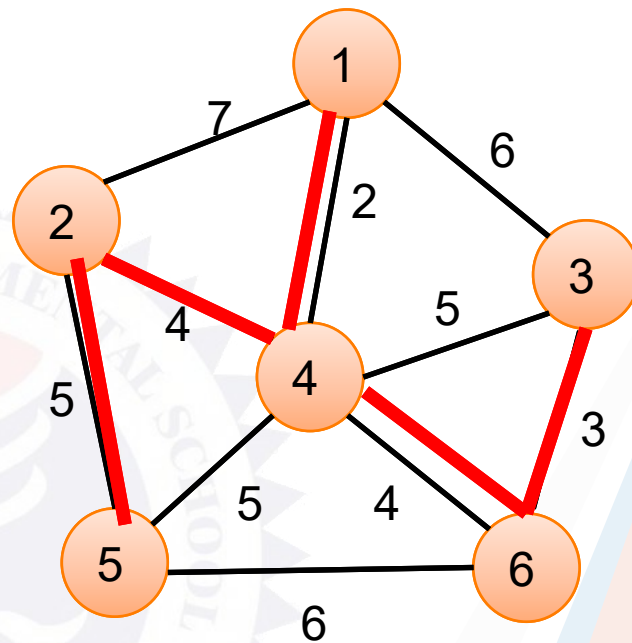
- ✧ 衡量一个点离生成树的远近，只看该点离生成树的最短边
- ✧ 当有新的点加入生成树后，会影响其它未加入点到生成树的距离。



→ Prim算法

实例过程分析：

- 1、把**1号**结点加入生成树
- 2、找到此时离生成树**最近距离为2**的点**4号**加点加入生成树。
- 3、找到此时离生成树**最近距离为4**的点**2号**加点加入生成树。
- 4、找到此时离生成树**最近距离为4**的点**6号**加点加入生成树。
- 5、找到此时离生成树**最近距离为3**的点**3号**加点加入生成树。
- 6、找到此时离生成树**最近距离为5**的点**3号**加点加入生成树。





Prim算法

```
int dist[maxn]//dist[i]表示i点离生成树的距离
bool vis[maxn];//vis[i]表示是否已加入生成树中
ans=0;//最小生成树的最小代价
vis[1]=true; dist[1]=0; //1号结点加入生成树,且更新距离
for(int i=head[1];i;i=edge[i].next){//枚举因为1号结点引起相邻结点到生成树距离降低的节点
    int v=edge[i].to;
    if(!vis[v] && dist[v]>edge[i].w) dist[v]=edge[i].w;//更新离生成树的距离。
}
for(int i=1;i<n;i++){//枚举加入其他n-1个节点
    int minVal=inf,minNode;
    for(int i=1;i<=n;i++){//找离生成树最近的点
        if(!vis[i] && minVal>dist[i]){
            minVal=dist[i]; minNode=i;
        }
    }
    ans+=minVal;vis[minNode]=true;
    for(int i=head[minNode];i;i=edge[i].next){
        int v=edge[i].to;
        if(!vis[v] && dist[v]>edge[i].w) dist[v]=edge[i].w;
    }
}
printf("%d",ans);
```


→ Kruskal算法

✧ **基本思想：**按照边的权值贪心，向生成树中加边

✧ **实现步骤：**

- 1.按照输入的边，建立边集数组，对边集数组按权值从小到大排序
- 2.把 n 个点放在生成树中，使得他们各种处于不同集合
- 3.向生成树中从最小的边开始加边。判断该边的两个顶点是否在同一集合中(表示**两个点不同时在生成树中**)，如果不在则添加
- 4.重复第3步，直到添加 $n-1$ 条边，则完成最小生成树的建立

✧ **注意：**

如何判断两个顶点是否在同一集合中呢？ **并查集的知识**



→ 并查集

✧ 问题提出：

给出各个元素之间的联系，要求将这些元素分成几个集合，每个集合中的元素直接或间接有联系。

在这类问题中主要涉及的是对集合的合并和查找，因此将这种问题称为并查集。

并查集共有四种操作

- 1 集合的建立
- 2 集合的元素的查询
- 3 集合的合并
- 4 集合的删除（较难，暂时不做讨论）

→ 并查集实现

集合建立: `father[]`---`Father[i]=i`

元素所在集合的查询:

```
int find(int x){  
    if(father[x]==x) return x;  
    else return find(father[x]);  
}
```

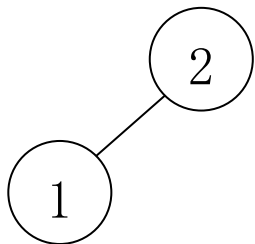
分析查询的时间复杂度？如何优化？



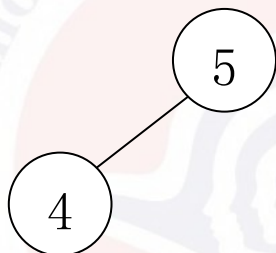
并查集分析优化



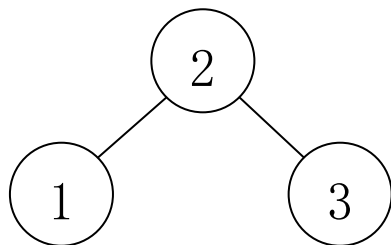
1 合并1和2



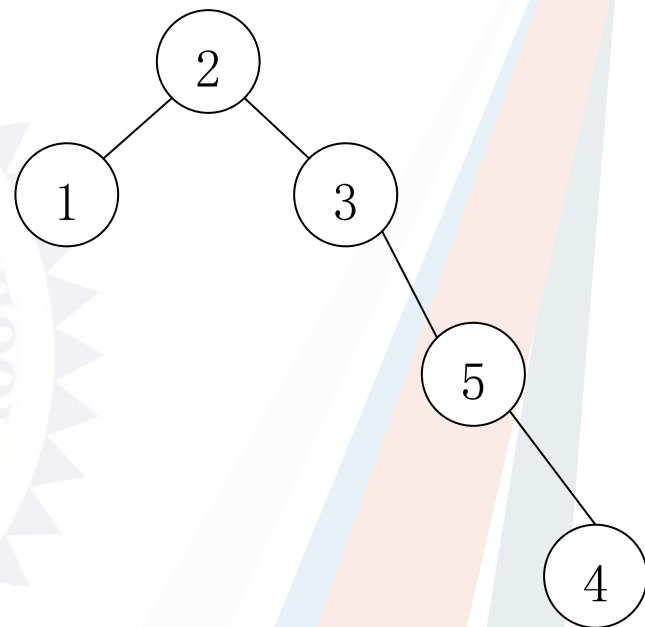
3 合并5和4



2 合并1和3



4 合并5和3



弊端!!!

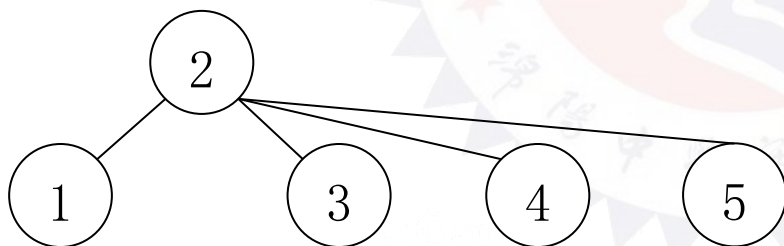
查询4所在的集合时间复杂度
如何修改呢?

→ 并查集---路径压缩

上述做法是指就是将元素的父亲结点指来指去的在指，当这棵树是链的时候，可见判断两个元素是否属于同一集合需要 $O(n)$ 的时间，于是路径压缩产生了作用。

路径压缩实际上是在找完根结点之后，在递归回来的时候顺便把路径上元素的父亲指针都指向根结点。

这就是说，我们在“合并5和3”的时候，不是简单地将5的父亲指向3，而是直接指向根节点1，如图





→ 路径压缩实现

元素所在集合的查询:

```
int find(int x)
{
    if(father[x]==x)
    {
        return x;
    }
    father[x]=find(father[x]);
    return father[x];
}
```



→ 并查集实现

集合的合并:

```
void merge(int x,int y)
{
    int fx=find(x);
    int fy=find(y);
    if(fx!=fy)
    {
        father[fx]=fy;
    }
}
```

注意：
这里不是x和y直接合并，而是x和y的祖先进行合并。
思考为什么？



→ 并查集扩展

并查集一般作用：

- 1、维护节点之间的连通性及求联通快
- 2、动态维护许多具有传递性的关系

并查集实际上是有若干颗树构成的森林，我们可以在树中的每条边上记录一个权值，即维护一个数组 d ，用 $d[x]$ 保存节点 x 到父节点 $fa[x]$ 之间的边权。在每次路径压缩后，每个访问过的节点就会直接指向树根，如果我们同时更新这些节点的 d 值，就可以利用路径压缩过程来统计每个节点到树根之间路径上的一些信息。这就是所谓的“**边带权**”的并查集。

➔ [NOI2002]银河英雄传说

✧ 问题描述:

有一个划分成 N 列的星际战场,各列依次编号为 $1,2,\dots,N$ 。有 N 艘战舰,也依次编号为 $1,2,\dots,N$,其中第 i 号战舰处于第 i 列,共有 M 条指令:

1 M_{ij} : 表示让第 i 号战舰所在列的全部战舰保持原有顺序,接在第 j 号战舰所在列的尾部,并且保证第 i 号战舰与第 j 号战舰不在同一列。

2 C_{ij} : 表示询问第 i 号战舰与第 j 号战舰是否处于同一列中,如果在同一列中,他们之间间隔了多少艘战舰?

✧ 数据范围: $N \leq 30000, m \leq 5 \times 10^5$

➔ 问题分析

利用并查集来处理：

一开始把 N 个战舰看成 N 个独立的集合。在没有路径压缩的情况下， $fa[x]$ 表示排在第 x 号战舰前面的那个战舰的编号。一个集合的代表(集合的根)就是位于最前面的战舰。另外，让树上每条边带权值为1，这样树上两点之间的距离减1就是二者之间间隔的战舰数量。

在考虑路径压缩的情况下，我们额外建立一个数组 d ， $d[x]$ 记录战舰 x 与 $fa[x]$ 之间的边的权值。在路径压缩把 x 直接指向树根的同时，我们把 $d[x]$ 更新为从 x 到树根的路径上的所有边权和。

并查集实现

查询：对d数组的维护

```
int find(int x){
    if(x==fa[x]) return x;
    int root=find(fa[x]);
    d[x]+=d[fa[x]];
    return fa[x]=root;
}
```

对于C指令，分别进行find(x)和find(y)完成查询和路径压缩，若二者的返回值相同，这说明x和y处于同一列中。所以d[x]保存了位于x之前的战舰数量，d[y]保存了位于y之前的战舰数量。二者只差减1，就是x和y之间间隔的战舰数量

合并：对d数组和size数组的维护

```
void merge(int x,int y){
    x=find(x);y=find(y);
    fa[x]=y;d[x]=size[y];
    size[y]+=size[x];
}
```

对于M指令，把x的树根作为y的树根的子节点，连接新边的权值应该为合并之间集合y的大小(依据题意，集合y中全部战舰都排在集合x之前)。因此，我们还需要一个size数组来记录每个树根上集合的大小。



Kruskal代码实现

```
for(int i=1;i<=m;i++){
    scanf("%d%d%d",&edge[i].u,&edge[i].v,&edge[i].w);
}
sort(edge+1,edge+1+m,cmp);
for(int i=1;i<=n;i++) fa[i]=i;
for(int i=1;i<=m;i++){
    if(find(edge[i].u)!=find(edge[i].v)){
        merge(edge[i].u,edge[i].v);
        ans+=edge[i].w; cnt++;
        if(cnt>=n-1) break;
    }
}
```



→ 并查集练习题目

P2024 食物链

P1682 过家家



→ 最小生成树的变形

✧ 最小生成树的变种：

- **基本概念：** 将一个图G的所有生成树边权值和按照从小到大排序，第K小的生成树称为K小生成树，其中**第二小生成树称为次小生成树**。
- **交换：** 图G的生成树T删除一条边并增加一条边的操作称为交换。
- 如果图G的两颗生成树可以通过一次交换得到，则这两颗生成树互为邻树。如果生成树T不是生成树集合S中的树，但T和S中的某颗树为邻树，则称T为S的邻树。
- 如果一个图G的**生成树在某些节点上有度的上限，称为限度生成树**。限度生成树中权值最小的生成树，称为最小限度生成树。

✧ 性质：

- 1.K小生成树不唯一。
- 2.设 $T_1, T_2, \dots, T_k$ 是图G的前K小生成树，则生成树集合 $\{T_1, T_2, \dots, T_k\}$ 的邻树种权值最小的树可作为K+1小生成树。
- 3.次小生成树一定是最小生成树的邻树。

➔ 次小生成树(P4180)

次小生成树又分为严格次小生成树和非严格次小生成树，
严格次小生成树，就是说：如果最小生成树选择的边集是EM，严格次小生成树选择的边集是ES，那么需满足：

$$\sum_{e \in EM} value(e) < \sum_{e \in ES} value(e)$$

次小生成树求法，算法流程：

- 1、求出任意一颗最小生成树T
- 2、枚举不在T中的一条边 $e(a,b)$ ，尝试用 e 替换最小生成树中 (a,b) 两点路径上的最大边（倍增实现）。
- 3、求出最小的替换代价即得到次小生成数的权值。

所需算法：kruksal+倍增，时间复杂度($E \cdot \log v$)



→ 限度生成树

Picnic Planning(POJ 1639)

给定一张N个点M条边的无向图，求出无向图的一颗最小生成树，满足1号节点的度数不超过给定的整数S。

数据范围： $N \leq 30$



限度生成树

限度生成树算法流程：

1. 忽略根节点做一次kruskal，此时得到的是一个森林，包含了 m 个最小生成树。

2. 对于每一颗最小生成树，选择其中**离根节点最近的点**，向根节点连一条边，此时得到了一棵 m 度的最小生成树。

3. 由 m 度生成树得到 $m+1$ 度生成树：

(1).用dp预处理出当前生成树中从 V_0 到点 i 的路径上与 V_0 无关联的权值最大的边,记为 $dp[i].d$,该边的两端点记为 $dp[i].u$ 和 $dp[i].v$ 。

(2).对于一个不在生成树中的边 $\langle V_0, v \rangle$ ，如果将该边加入生成树中，则一定会得到一个环。此时我们删掉环中权值最大的边，即(1)中预处理得到的 $dp[v]$,得到一棵 $m+1$ 度的生成树。

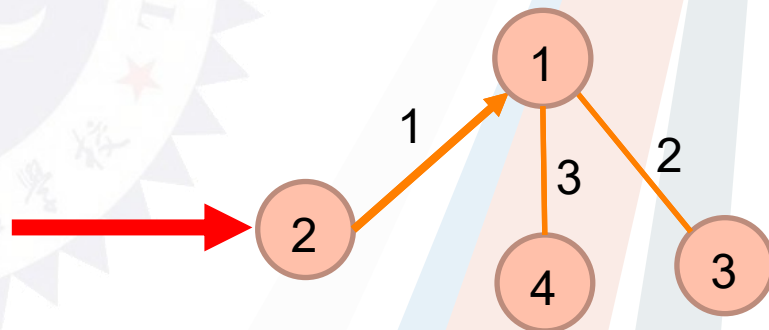
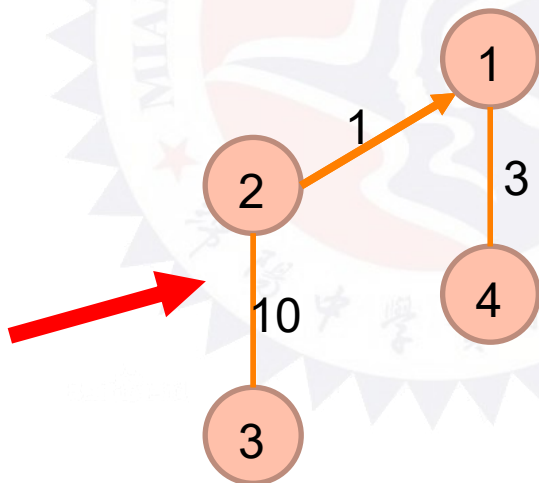
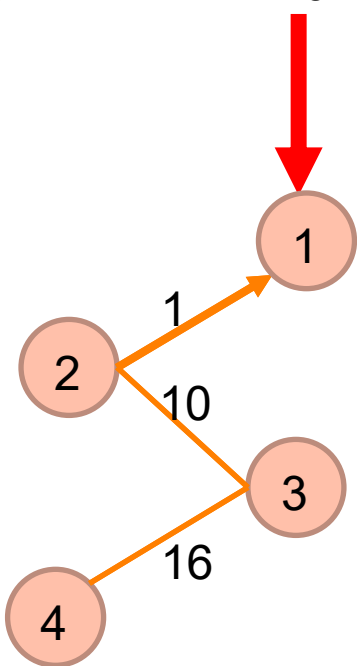
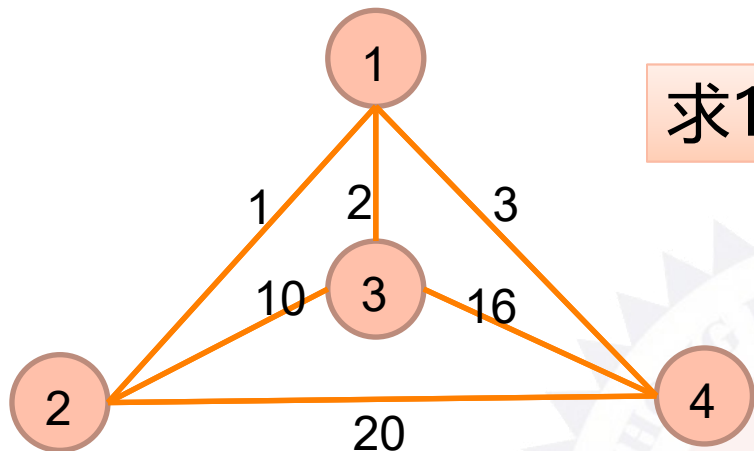
(3).对于(2)枚举每一个 v ，记 $minnum = \min\{(V_0, v) - dp[v].d\}$ ，使 $minnum$ 得到最小值的点 v 就是这次选择的点。连接 (V_0, v) ，删去 $dp[v]$ 。



限度生成树

求1号节点度小于等于3的最小生成树

总体时间复杂度: N^4





→ 最优比率生成树

给定一张 N 个点， M 条边的无向图($1 \leq N, M \leq 1000$), 图中每条边 e 都有一个收益 C_e 和一个成本 R_e 。求该图的一颗生成树 T , 使树中各边的收益之和除以成本之和, 即 $\sum_{k=1}^m C_e / \sum_{k=1}^m R_e$ 最大。



→ 0/1分数规划

01分数规划问题：所谓的01分数规划问题就是指这样的一类问题，给定两个数组， $a[i]$ 表示选取 i 的收益， $b[i]$ 表示选取 i 的代价。如果选取 i ，定义 $x[i]=1$ 否则 $x[i]=0$ 。每一个物品只有选或者不选两种方案，求一个选择方案使得 $R=\sigma(a[i]*x[i])/\sigma(b[i]*x[i])$ 取得最值，即所有选择物品的总收益/总代价的值最大或是最小。

0/1分数规划模型是指，给定整数 $a_1, a_2, \dots, a_n$ 以及 $b_1, b_2, \dots, b_n$ ，求一组解 $x_i (1 \leq i \leq n, x_i = 0 \text{ or } 1)$ ，使得下式最大化：

$$\frac{\sum_{i=1}^n a_i * x_i}{\sum_{i=1}^n b_i * x_i}$$

通俗地说，就是给定 n 对整数 a_i, b_i ，从中选出若干对，使得选出的数对的 a 之和与 b 之和的商最大。

0/1分数规划

设一个值 L ，考虑是否存在一组解 $\{x_1, x_2, \dots, x_n\}$ 满足：

$$\sum_{i=1}^n (a_i - L * b_i) * x_i \geq 0$$

1、如果存在，对上式变形得： $\sum_{i=1}^n a_i * x_i - L * \sum_{i=1}^n b_i * x_i \geq 0$ ，可知：

存在 $\{x_1, x_2, \dots, x_n\}$ ，使得 $\frac{\sum_{i=1}^n a_i * x_i}{\sum_{i=1}^n b_i * x_i} \geq L$ ，也就是说， L 比我们求的最大值要小。

2、如果对于任意一组解，都有 $\sum_{i=1}^n (a_i - L * b_i) * x_i \leq 0$ ，同理可知，

任意 $\{x_1, x_2, \dots, x_n\}$ ，都有 $\frac{\sum_{i=1}^n a_i * x_i}{\sum_{i=1}^n b_i * x_i} < L$ ，也就是说， L 比我们求的最大值要大。

这与“二分答案”的情形非常相似。虽然最终答案未知，但是如果我们猜测一个值 L ，就能通过判定“是否存在一组满足 $\sum_{i=1}^n (a_i - L * b_i) * x_i \geq 0$ ”得到 L 应该变得更大还是更小。也就是， L 关于“解的存在性”具有单调性。

如何判定“是否存在一组解满足 $\sum_{i=1}^n (a_i - L * b_i) * x_i \geq 0$ ”呢？显然只需要判定表达式的最大值是否非负即可，这就等价于从 n 个数里选出若干个，使得他们的和最大。把所有的证书加起来即可。



→ 0/1分数规划

综上所述，我们可以二分答案（实数）。当二分的值为mid时，我们就计算 $\sum_{i=1}^n (a_i - L * b_i) * x_i$ 的最大值，检查最大值是否非负。若非负，责令 $L=mid$ ，否则令 $r=mid$ 。当二分停止时，就得到了0/1分数规划问题的解。

除二分法外，0/1分数规划有另外求法—**Dinkelbach迭代法**

<https://blog.csdn.net/hehaitao074/article/details/11562069>

<https://blog.csdn.net/chenzhenyu123456/article/details/48160209>

→ 最优比率生成树

根据0/1分数规划问题，我们只需构建一张新的无向图，图的结构不变，但是每条边只有一个权值 $C_e - \text{mid} * R_e$ 。在新的无向图中求最大生成树，若最大生成树上边权之和非负，则令 $l = \text{mid}$ ，否则令 $r = \text{mid}$ 。当二分结束 l 即为答案。

注：实数二分过程如下

```
#define eps 1e-8
double binary_search()
{
    double l = 0, r = Max, mid;
    while(r - l >= eps)
    {
        mid = (l + r) / 2;
        if(judge(mid))
            l = mid;
        else
            r = mid;
    }
    return l;
}
```

→ 最短路径生成树

黑暗城堡(`castle.cpp`)

在顺利攻破 Lord lsp 的防线之后，lqr 一行人来到了 Lord lsp 的城堡下方。Lord lsp 黑化之后虽然拥有了强大的超能力，能够用意念力制造建筑物，但是智商水平却没怎么增加。现在 lqr 已经搞清楚黑暗城堡有 N 个房间， M 条可以制造的双向通道，以及每条通道的长度。

lqr 深知 Lord lsp 的想法，为了避免每次都要琢磨两个房间之间的最短路径，Lord lsp 一定会把城堡修建成树形的；但是，为了提高自己的移动效率，Lord lsp 一定会使得城堡满足下面的条件：

设 D_i 为如果所有的通道都被修建，第 i 号房间与第 1 号房间的最短路径长度；而 S_i 为实际修建的树形城堡中第 i 号房间与第 1 号房间的路径长度，对于所有满足 $1 \leq i \leq N$ 的整数 i ，有 $S_i = D_i$ 。

为了打败 Lord lsp，lqr 想知道有多少种不同的城堡修建方案。于是 lqr 向 applepi 提出了这个问题。由于 applepi 还要忙着出模拟赛，所以这个任务就交给你了。当然，你只需要输出答案对 $2^{31} - 1$ 取模之后的结果就行了。

→ 最短路径生成树

很显然题目要求的是一个图的生成树，这棵生成树要求满足根节点到每个结点的距离都等于原图中的最短距离，我们不妨将这样的树称之为这个图的最短路径生成树
首先生成树中的边肯定是最短路径子图中的边

我来解释一下这个概念.....

首先图是由点集和边集两个元素组成的

那么子图的点集和边集肯定是原图中的子集

最短路径子图的点集肯定和原图一样

顾名思义边集就是所有可能出现在最短路径中的边的集合

即 $E'=\{e|\text{dis}[e.u]+e.l=\text{dis}[e.v]\}$

我们可以用显然法证明，最短路径生成树一定是最短路径子图的生成树

然而最短路径子图的生成树却并不一定是最短路径生成树

举个例子

$V=\{1,2,3\}, E=\{(1,2,1),(2,3,1),(1,3,2)\}$ (最后一个元素代表边的长度)

显然这个图的每一条边都可能出现在最短路径中

所以这个图的最短路径子图就是它本身

$\{(1,3,2),(2,3,1)\}$ 是这个图的最短路径子图的一个生成树

然而并不是这个图的最短路径生成树，因为1到2的距离是3

有了以上的思路之后，我们可以先求出来最短路径子图

→ 最短路径生成树

- ✧ 最短路径子图怎么求？
- ✧ 根据定义，跑一遍任何一个单源最短路算法(dij, SPFA)，然后 $dis[i] + w[i][j] = dis[j]$ 的边就是最短路径子图的边
- ✧ 然后将这些点按dis排序
- ✧ 假设我们现在已经求出了连通 $1 \dots i-1$ 这几个点的最短路径生成树个数
- ✧ 首先第 i 个点肯定是由 $1 \dots i-1$ 中的其中一个点连接的，而不可能是dis比 i 还大的那些点连接过来的。
- ✧ 然后 $1 \dots i-1$ 的生成树的形态和 $1 \dots i-1$ 中哪个点连接 i 这个点是没有关系的
- ✧ 也就是说这两个方案是相互独立的，计数中的互相独立意味着乘法原理的可行性
- ✧ 那么连通 $1 \dots i$ 的最短路径生成树个数就是 $1 \dots i-1$ 的生成树个数乘以 $1 \dots i-1$ 中任何一个点连接 i 这个点的最短路边的个数

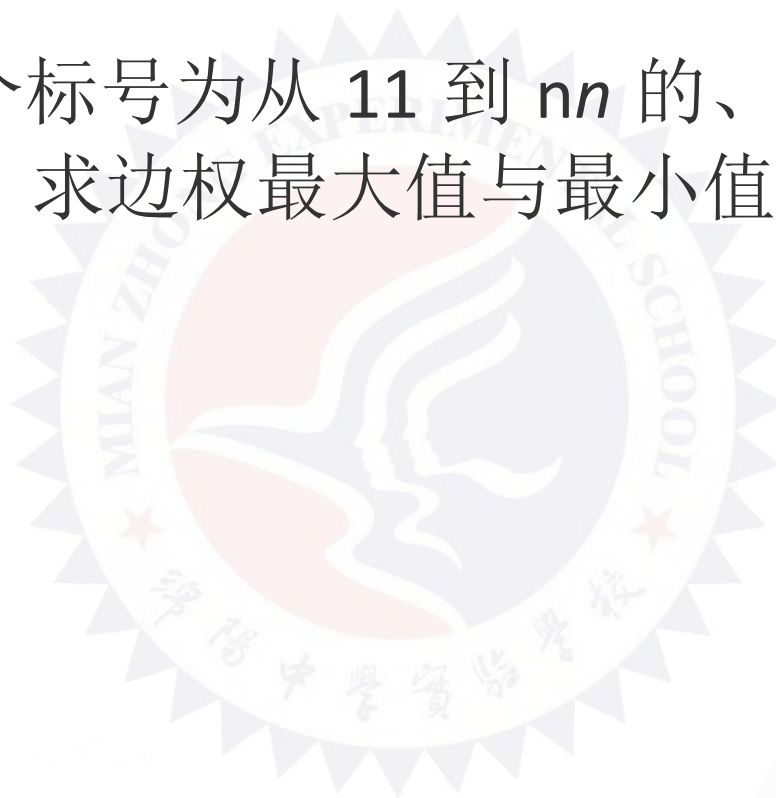


→ 最小差值生成树

落谷： P4234

题目描述

给定一个标号为从 1 到 n 的、有 m 条边的无向图，求边权最大值与最小值的差值最小的生成树。



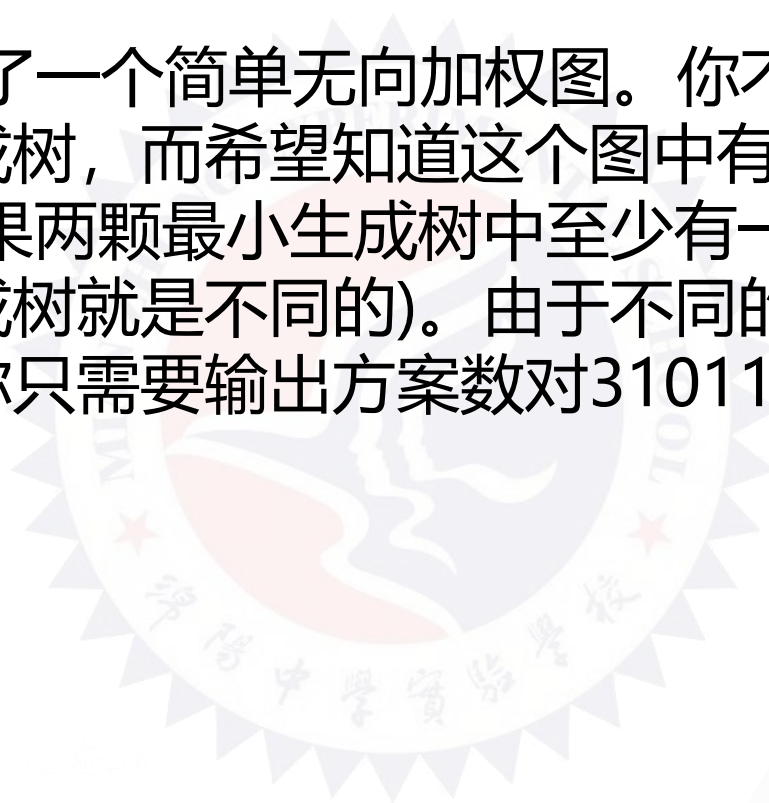


→ 最小生成树计数

落谷：P4208 [JSOI2008]

题目描述

现在给出了一个简单无向加权图。你不满足于求出这个图的最小生成树，而希望知道这个图中有多少个不同的最小生成树。(如果两颗最小生成树中至少有一条边不同，则这两个最小生成树就是不同的)。由于不同的最小生成树可能很多，所以你只需要输出方案数对31011的模就可以了。





→ 并查集

P0J1456 (Supermarket)

给定 N 个商品，每个商品有利润 P_i 和过期时间 D_i ，每天只能卖一个商品，过期的商品不能再卖，求如何安排每天卖的商品，可以使收益最大。

数据范围： $1 \leq N, P_i, D_i \leq 10000$ 。

问题分析：

做法一：用堆(优先队列)

做法二：贪心

The background features a central point from which several wide, colorful rays emanate, creating a starburst effect. The rays are in shades of blue, orange, red, and grey. Faint, scattered binary code (0s and 1s) is visible in the background, particularly on the right side.

Thank you!